From nothing to a modal editor

Abstract

We grow a small terminal editor from a program that does nothing, then give it insert and command modes. The construction follows one equation: inserting two lists in succession is insertion of their concatenation. A zipper makes this law easy to prove; a keyboard interpretation turns it into a law about typing; adding modes turns it into a law about complete editing sessions. The programs and their proofs are written in Lean 4. Along the way, we distinguish equality of editor states from correctness of terminal I/O.

1 Begin by returning

Our first program is easy to leave.

```
def main : IO Unit := pure ()
```

It returns without reading a key or displaying a character. We keep it as the first of three executables: a program that does nothing, an editor in which typing inserts text, and a modal editor in which a letter can instead name a command. The useful steps should preserve something of this beginning's simplicity.

There are two decisions to make. We need a place at which to edit, and a meaning for each key. We shall first settle the place, then give keys one meaning, and finally allow a mode to choose between two meanings. Our destination is a law for the interaction "enter insert mode, type some text, return to command mode". Splitting the text across two such interactions will have the same final effect as entering it in one.

The representation is the list form of Huet's zipper (Huet, 1997); the style of development owes much to Bird's calculation of a Sudoku solver (Bird, 2006). We use Lean 4 (de Moura and Ullrich, 2021) both to run the editor and to check its equations. Familiarity with functional programming and lists will suffice for the argument. The few proof scripts we display are accompanied by their calculations.

2 Find a place

A document and a natural number would give us text and a cursor, provided the number stayed within the document. We could carry a proof of that condition. Instead, let the cursor be a cut in the text. Store the part before the cut backwards and the part after it forwards:

$$\texttt{hel}\,|\,\texttt{lo} \qquad \longleftrightarrow \qquad \langle [\texttt{l},\texttt{e},\texttt{h}], [\texttt{l},\texttt{o}] \rangle .$$

The next character in either direction is now at the head of a list.

```
structure Buffer where
  before : List Char := []
  after : List Char := []
```

Every pair of lists is a valid buffer, including the pair of empty lists. The focus is a gap: there are $n + 1$ possible cursor positions in a document of n characters. This convention will remain the same in both modes.

We observe a buffer in two ways:

```
def text (b : Buffer) : List Char := b.before.reverse ++ b.after
def position (b : Buffer) : Nat := b.before.length
def ofText (cs : List Char) : Buffer := ⟨[], cs⟩
```

The functions live in the `Buffer` namespace; `b.text` abbreviates `Buffer.text b`. Angle brackets construct a record, and `++` concatenates lists. We omit surrounding namespaces in the listings.

Write $\mathsf{T}b$ for the text and $\mathsf{P}b$ for the position. For $b = \langle l, r \rangle$, the definitions give

$$\mathsf{T}b = \operatorname{rev} l \mathbin{+\!\!+} r, \qquad \mathsf{P}b = |l| \le |\mathsf{T}b|. \tag{1}$$

Taking and dropping at the cursor recovers the two pieces:

$$\operatorname{take}(\mathsf{P}b, \mathsf{T}b) = \operatorname{rev} l, \qquad \operatorname{drop}(\mathsf{P}b, \mathsf{T}b) = r. \tag{2}$$

Reversal is its own inverse, so text and position determine the entire buffer. This gives an extensionality principle: to specify an edit completely, specify its resulting text and position. The representation has neither an out-of-bounds cursor nor an unobservable extra field.

Moving the gap transfers a head between the lists.

```
def left : Buffer → Buffer
  | ⟨[], cs⟩ => ⟨[], cs⟩
  | ⟨c :: cs, rest⟩ => ⟨cs, c :: rest⟩

def right : Buffer → Buffer
  | ⟨cs, []⟩ => ⟨cs, []⟩
  | ⟨cs, c :: rest⟩ => ⟨c :: cs, rest⟩
```

An impossible move leaves the buffer alone. Both operations preserve T; their positions are respectively $\mathsf{P}b - 1$ and $\min(\mathsf{P}b + 1, |\mathsf{T}b|)$, where subtraction on natural numbers saturates at zero. Thus

$$(b.\texttt{left}).\texttt{right} = b \quad \text{if } 0 < \mathsf{P}b.$$

The hypothesis is essential. At the beginning of a nonempty document, left does nothing and right advances. Total functions have not made the boundary disappear; they have made its behaviour explicit.

Now insert a list before the gap, leaving the gap after the new text. Backspace and forward delete discard a head on the appropriate side.

```
def insert (b : Buffer) (cs : List Char) : Buffer :=
  ⟨cs.reverse ++ b.before, b.after⟩

def backspace (b : Buffer) : Buffer := ⟨b.before.tail, b.after⟩
def delete (b : Buffer) : Buffer := ⟨b.before, b.after.tail⟩
```

Lean defines the tail of an empty list to be empty. Deletion therefore needs no exceptional state. Inserting a character and then backspacing restores the original buffer exactly.

The specification of insertion need not mention reversed lists. If $t = \mathsf{T}b$ and $p = \mathsf{P}b$, it is

$$\mathsf{T}(b.\texttt{insert}\ xs) = \mathrm{take}(p, t) +\!\!+ xs +\!\!+ \mathrm{drop}(p, t), \tag{3}$$

$$\mathsf{P}(b.\texttt{insert}\ xs) = p + |xs|. \tag{4}$$

Substituting (2), the first equation reduces to

$$\mathrm{rev}(\mathrm{rev}\, xs +\!\!+ l) +\!\!+ r = \mathrm{rev}\, l +\!\!+ xs +\!\!+ r.$$

Reverse an append, cancel two reversals, and reassociate. The cursor equation uses the lengths of reversal and append. By extensionality, these two calculations account for the whole edit. The accompanying code checks them as `text_insert` and `position_insert`; backspace and delete have analogous `take`/`drop` specifications.

3 Put two insertions together

Write $b \triangleleft xs$ for `b.insert xs`. For $b = \langle l, r\rangle$, expanding the definition gives

$$\begin{aligned}(b \triangleleft xs) \triangleleft ys &= \langle \mathrm{rev}\, ys +\!\!+ (\mathrm{rev}\, xs +\!\!+ l), r\rangle \\ &= \langle \mathrm{rev}(xs +\!\!+ ys) +\!\!+ l, r\rangle \\ &= b \triangleleft (xs +\!\!+ ys).\end{aligned}$$

The second block is nearer the head of the reversed list, hence later in the document. Together with insertion of the empty list, we have our first law.

Law 1 (Insertion)**.** *For every buffer b and character lists xs, ys,*

$$b \triangleleft [] = b, \qquad (b \triangleleft xs) \triangleleft ys = b \triangleleft (xs +\!\!+ ys). \tag{5}$$

In algebraic language, insertion is a right action of the monoid of lists under concatenation. The practical content is that a typing task may be divided into pieces without changing its final result. This is a law of whole buffers, including their cursor positions.

The concatenation equation takes one simplification in Lean:

```
theorem insert_append (b : Buffer) (xs ys : List Char) :
    (b.insert xs).insert ys = b.insert (xs ++ ys) := by
  simp [insert, List.reverse_append, List.append_assoc]
```

Here `simp` unfolds insertion and applies the stated list identities. The calculation explains why those identities are sufficient. Later we will need exactly this law at a larger scale.

4 Give the keyboard a meaning

A buffer can be edited, but cannot yet be used. A terminal reader will give us values of type `Key`: characters, arrows, Enter, Backspace, and the other decoded inputs. Each key chooses an action. Most actions transform the state; two ask the surrounding program to do something else.

```
inductive Action (State : Type) where
  | edit (change : State → State)
  | save | quit
```

For the moment, `State` is `Buffer`. Keeping it a parameter will cost us nothing now and save us work when modes arrive. The state transformation performed by an action is

```
def Action.apply (action : Action State) (s : State) : State :=
  match action with
  | .edit change => change s
  | .save | .quit => s
```

This function is a projection onto the pure part of an action. The terminal loop must also inspect the constructor to carry out a save or quit request. Neither request changes the buffer value.

The first interpretation of the keyboard inserts characters and assigns the remaining keys their direct editing operations:

```
def Key.action : Key → Action Buffer
  | .character c => .edit (·.insert [c])
  | .paste cs => .edit (·.insert cs)
  | .left => .edit Buffer.left
  | .right => .edit Buffer.right
  | .up => .edit Buffer.up
  | .down => .edit Buffer.down
  | .home => .edit Buffer.home
  | .lineEnd => .edit Buffer.lineEnd
  | .delete => .edit Buffer.delete
  | .backspace => .edit Buffer.backspace
  | .enter => .edit (·.insert ['\n'])
  | .tab => .edit (·.insert ['\t'])
  | .save => .save
  | .quit => .quit
  | .escape | .redraw => .edit id
```

The section `(·.insert [c])` means `fun b => b.insert [c]`. Line movements are additional buffer functions built from `seek`, which cuts the text at a requested offset and reconstructs its zipper. Their formulas will play no role in the modal extension: any function from `Buffer` to `Buffer` can be bound in the same way.

We can now close a loop: draw the buffer, read a key, choose an action, apply its pure part, and handle any request. Ctrl-S denotes save and Ctrl-Q denotes quit. Remembering the text after a successful save lets the loop recognize unsaved changes. Cursor movement cannot make a clean file dirty, by preservation of `T`.

This is our second executable, `walk-editor`. It opens a file in a terminal, accepts typing, moves the cursor, scrolls, and saves. It has one interpretation of the keyboard and no need to consult a mode. We have reached an editor before designing a command language.

5 Give the same key another meaning

Suppose `h` should also mean "move left". A function from keys to actions cannot choose between that command and insertion of the letter. We need one more input to the choice: a mode.

```
inductive Mode where
  | insert | command
  deriving DecidableEq, Repr

structure Editor where
  buffer : Buffer
  mode : Mode := .command
```

The buffer is unchanged. We lift its operations to the larger state by changing one field and preserving the other:

```
def change (f : Buffer → Buffer) (e : Editor) : Editor :=
  { e with buffer := f e.buffer }
def enter (mode : Mode) (e : Editor) : Editor := { e with mode }
```

These functions live in the `Editor` namespace. Record update exposes the independence of the two coordinates. In particular,

$$\mathsf{change}\ g\ (\mathsf{change}\ f\ e) = \mathsf{change}\ (g \circ f)\ e.$$

Lifting buffer operations respects their order of composition. The corresponding lifting of actions keeps the two requests intact:

```
def lift : Action Buffer → Action Editor
  | .edit f => .edit (change f)
  | .save => .save
  | .quit => .quit
```

The useful commuting equation is small enough to prove by inspection:

```
theorem apply_lift (a : Action Buffer) (b : Buffer) (m : Mode) :
    (lift a).apply ⟨b, m⟩ = ⟨a.apply b, m⟩ := by
  cases a <;> rfl
```

Inspect the three constructors of `a`; each case reduces to equality of the same value. Applying an old action and adjoining the mode agrees with adjoining the mode and applying the lifted action. This is the reason the old editor can be reused inside the new one.

Insert mode will use the old interpretation. Command mode supplies a new one:

```
def command : Key → Action Editor
  | .character 'i' => .edit (enter .insert)
  | .character 'a' => .edit (enter .insert ∘ change Buffer.right)
  | .character 'h' => lift (.edit Buffer.left)
  | .character 'j' => lift (.edit Buffer.down)
  | .character 'k' => lift (.edit Buffer.up)
  | .character 'l' => lift (.edit Buffer.right)
  | .character '0' => lift (.edit Buffer.home)
  | .character '$' => lift (.edit Buffer.lineEnd)
  | .character 'x' => lift (.edit Buffer.delete)
  | .character 'w' => .save
  | .character 'q' => .quit
  | .character _ | .paste _ | .enter | .tab | .backspace => .edit id
  | key => lift key.action
```

The interesting new actions are `i`, which changes only the mode, and `a`, which composes a rightward move with that change. Unbound characters do nothing. Physical navigation keys retain their old meanings. Finally, the mode selects the interpretation, with Escape always returning to command mode:

```
def action (e : Editor) (key : Key) : Action Editor :=
  match key with
  | .escape => .edit (enter .command)
  | _ => match e.mode with
    | .insert => lift key.action
    | .command => command key
```

Escape leaves the gap in place. This convention matters: changing mode preserves both text and cursor, and a second Escape has no further effect. It will let us state an exact round trip without a cursor correction.

The third executable, `lean-editor`, starts in command mode. Press `i`, type `hello`, and press Escape. Then press `0`, `x`, `i`, `H`, Escape, `w`, and `q`. Starting with an empty file, this saves `Hello`. Both editors use the same terminal loop: the direct editor always lifts `Key.action`; the modal editor consults `Editor.action`. In the shared implementation the direct editor's mode field is fixed and ignored by its bindings.

6 Recover the insertion law

We have changed the interface substantially. What remains of the insertion law? Define one pure transition, then thread it through a list of keys:

```
def step (e : Editor) (key : Key) : Editor := (e.action key).apply e

def replay (e : Editor) (keys : List Key) : Editor := keys.foldl step e
```

Write $\mathsf{R}(e, ks)$ for `replay e ks`. The equations of a left fold are

$$\begin{aligned} \mathsf{R}(e, []) &= e, \\ \mathsf{R}(e, k :: ks) &= \mathsf{R}(\texttt{step}\ e\ k, ks), \\ \mathsf{R}(e, ks \mathbin{+\!\!+} ls) &= \mathsf{R}(\mathsf{R}(e, ks), ls). \end{aligned} \tag{6}$$

This describes state changes. It does not stop at a quit request or write a file at a save request, since `Action.apply` projects both to the identity. Consequently these equations do not claim equality of complete I/O executions. The insert sessions below contain neither request.

We can first state precisely how insert mode reuses the direct editor. Let $\mathsf{D}(b, ks)$ be the fold that applies `key.action.apply` to a buffer for each key. For any list ks containing no Escape,

$$\mathsf{R}(\langle b, \mathsf{insert}\rangle, ks) = \langle \mathsf{D}(b, ks), \mathsf{insert}\rangle. \tag{7}$$

For one key, `action` chooses `lift key.action`, so `apply_lift` gives the equality. Induction extends it to a list, using the equation again on the updated buffer. Escape is excluded exactly because it changes the mode. The checked theorem `replay_insert` covers movement, deletion, newlines, and pastes as well as ordinary characters; it is a statement about their pure states, including the state projections of save and quit.

Now let $\mathsf{chars}(xs)$ map `Key.character` over a character list. Typing that list in insert mode amounts to bulk insertion:

$$\mathsf{R}(\langle b, \mathsf{insert}\rangle, \mathsf{chars}(xs)) = \langle b \triangleleft xs, \mathsf{insert}\rangle. \tag{8}$$

For $xs = []$, this is insertion of the empty list. In the inductive case, expose the first character and apply the induction hypothesis:

$$\begin{aligned} \mathsf{R}(\langle b, \mathsf{insert}\rangle, \mathsf{chars}(c :: cs)) &= \mathsf{R}(\langle b \triangleleft [c], \mathsf{insert}\rangle, \mathsf{chars}(cs)) \\ &= \langle (b \triangleleft [c]) \triangleleft cs, \mathsf{insert}\rangle \\ &= \langle b \triangleleft (c :: cs), \mathsf{insert}\rangle. \end{aligned}$$

The last equality is the insertion law. Its use is visible in the Lean proof:

```
theorem type_text (b : Buffer) (cs : List Char) :
    replay ⟨b, .insert⟩ (cs.map Key.character) =
      ⟨b.insert cs, .insert⟩ := by
  induction cs generalizing b with
  | nil => simp
  | cons c cs ih =>
    rw [List.map_cons, replay_cons, step_insert_character, ih]
    simp [Buffer.insert_append]
```

The phrase `generalizing b` is important: the induction hypothesis must apply after insertion of the first character, when the buffer has changed.

Let `i` denote `Key.character 'i'`, let `esc` denote `Key.escape`, and package the three parts of an insert session as

$$\mathsf{S}(xs) = [\mathsf{i}] \mathbin{+\!\!+} \mathsf{chars}(xs) \mathbin{+\!\!+} [\mathsf{esc}]. \tag{9}$$

We can now close the round trip.

Law 2 (Insert session)**.** *For every buffer b and character list xs,*

$$\mathsf{R}(\langle b, \mathsf{command}\rangle, \mathsf{S}(xs)) = \langle b \triangleleft xs, \mathsf{command}\rangle. \tag{10}$$

Proof. The first key enters insert mode without changing the buffer. Equation (8) accounts for the character keys. The last key returns to command mode without changing the buffer. Splitting the replay at these boundaries gives the calculation

$$\begin{aligned}\mathsf{R}(\langle b, \mathsf{command}\rangle, \mathsf{S}(xs)) &= \mathsf{R}(\langle b, \mathsf{insert}\rangle, \mathsf{chars}(xs) \mathbin{+\!\!+} [\mathsf{esc}]) \\ &= \mathsf{R}(\langle b \triangleleft xs, \mathsf{insert}\rangle, [\mathsf{esc}]) \\ &= \langle b \triangleleft xs, \mathsf{command}\rangle.\end{aligned}$$

□

The corresponding proof contains only the rewrites just described:

```
theorem insert_session (b : Buffer) (cs : List Char) :
    replay ⟨b, .command⟩
      (.character 'i' :: (cs.map Key.character ++ [.escape])) =
        ⟨b.insert cs, .command⟩ := by
  rw [replay_cons, step_command_i, replay_append, type_text,
    replay_cons, step_escape]
  rfl
```

This is equality of the whole editor value. Equations (3) and (4) therefore give the final text and cursor, and the theorem supplies the final mode. The statement places no bound on the document, its cursor offset, or the inserted list.

There is a small surprise left. The encoding S does *not* preserve concatenation of character lists as concatenation of key lists: $\mathsf{S}(xs) \mathbin{+\!\!+} \mathsf{S}(ys)$ contains a middle Escape and `i` that $\mathsf{S}(xs \mathbin{+\!\!+} ys)$ lacks. After interpretation from command mode, however, those delimiters disappear:

$$\begin{aligned}&\mathsf{R}(\langle b, \mathsf{command}\rangle, \mathsf{S}(xs) \mathbin{+\!\!+} \mathsf{S}(ys)) \\ &\quad= \mathsf{R}(\langle b \triangleleft xs, \mathsf{command}\rangle, \mathsf{S}(ys)) \\ &\quad= \langle (b \triangleleft xs) \triangleleft ys, \mathsf{command}\rangle \\ &\quad= \langle b \triangleleft (xs \mathbin{+\!\!+} ys), \mathsf{command}\rangle \\ &\quad= \mathsf{R}(\langle b, \mathsf{command}\rangle, \mathsf{S}(xs \mathbin{+\!\!+} ys)).\end{aligned} \tag{11}$$

Equation (6), the session law, and the original insertion law are sufficient. The theorem `insert_sessions` checks the calculation. Insertion of the empty list becomes an empty session that leaves a command-mode editor unchanged. Complete insert sessions thus realize the same right action as bulk insertion, even though their encodings contain extra keys. They agree in final state; their intermediate displays and input histories still differ.

7 The terminal at the boundary

The objects in the session law are decoded events. A `Key.character` is data even when its character is `q` or `i`. Physical Enter, Tab, and Escape are different constructors. The theorem does not assert that any chosen list of raw terminal bytes decodes to `chars xs`.

Pasting makes the distinction useful. A bracketed paste arrives as one `Key.paste` carrying a list, rather than as a list of character commands. The mode-specific rules are

$$\texttt{step}\ \langle b, \mathsf{insert}\rangle\ (\texttt{paste}\ xs) = \langle b \triangleleft xs, \mathsf{insert}\rangle,$$
$$\texttt{step}\ \langle b, \mathsf{command}\rangle\ (\texttt{paste}\ xs) = \langle b, \mathsf{command}\rangle.$$

Both are checked by reduction. Thus a decoded paste containing `xwq` is inserted literally or ignored; it cannot delete, save, and quit. The reader implements xterm's bracketed-paste convention (Moy et al., 2026) and normalizes pasted CRLF and CR line endings to LF. Input without the paste delimiters is ordinary keyboard input.

The terminal code also resolves Escape's second role as an escape-sequence prefix, decodes UTF-8, draws a scrolling viewport, and restores raw settings on normal exit and caught I/O errors. The actual loop applies the same pure action as `step`. File writes, byte decoding, terminal cleanup, and glyph placement are outside the proofs. The library additionally proves seeking, text preservation for line movement, and the viewport bounds; it does not separately prove the exact row and column chosen by Up and Down.

Costs have a similarly clear boundary. Left, right, backspace, delete, and single-character insertion use constant-size list manipulations. Inserting k characters takes $O(k)$ such work. Observing text, calculating line motion, and redrawing can traverse the document. These are structural cost arguments, not formal resource bounds. The complete editor redraws on each key and is intended for small files. It assumes one screen cell per character, with tabs shown as four spaces; wide glyphs and combining sequences require more work. Saving overwrites the target directly. Undo, selection, and search are absent.

The accompanying artifact preserves all three stops:

```
lake build --wfail
lake exe walk-empty
lake exe walk-editor notes.txt
lake exe lean-editor notes.txt
```

The terminal programs require `stty` and a UTF-8 terminal on macOS or Linux. The repository pins Lean 4.34.1 and uses its standard library. Building checks the proofs and produces the executables; there are no project-specific axioms or `sorry` placeholders. The proof terms rely on Lean's trusted logical environment. They do not verify its compiler or the operating system.

The buffer definitions are in `LeanEditor/Basic.lean`; the two keyboard interpretations and interaction proofs are in `Keys.lean` and `Editor.lean` under the same directory. Every Lean listing in this paper is checked against the source when building the document. The definitions are the ones executed by the editor, with namespace wrappers omitted here.

We started with a program that could only return. A gap gave it a place to edit; functions gave its keys meanings; a second coordinate let those meanings change. The law that explained two insertions now explains two conversations with the editor. The extra interface has preserved the algebra we began with.

References

Bird, R. (2006). A program to solve Sudoku. *Journal of Functional Programming*, 16(6), 671–679. doi:10.1017/S0956796806006058.

de Moura, L. and Ullrich, S. (2021). The Lean 4 theorem prover and programming language (system description). In *Automated Deduction—CADE 28*, LNCS 12699, 625–635. doi:10.1007/978-3-030-79876-5_37.

Huet, G. (1997). The zipper. *Journal of Functional Programming*, 7(5), 549–554. doi:10.1017/S0956796897002864.

Moy, E., revised by Gildea, S. and Dickey, T. (2026). *XTerm Control Sequences*. Patch 411, 23 August 2026. https://invisible-island.net/xterm/ctlseqs/ctlseqs.html.